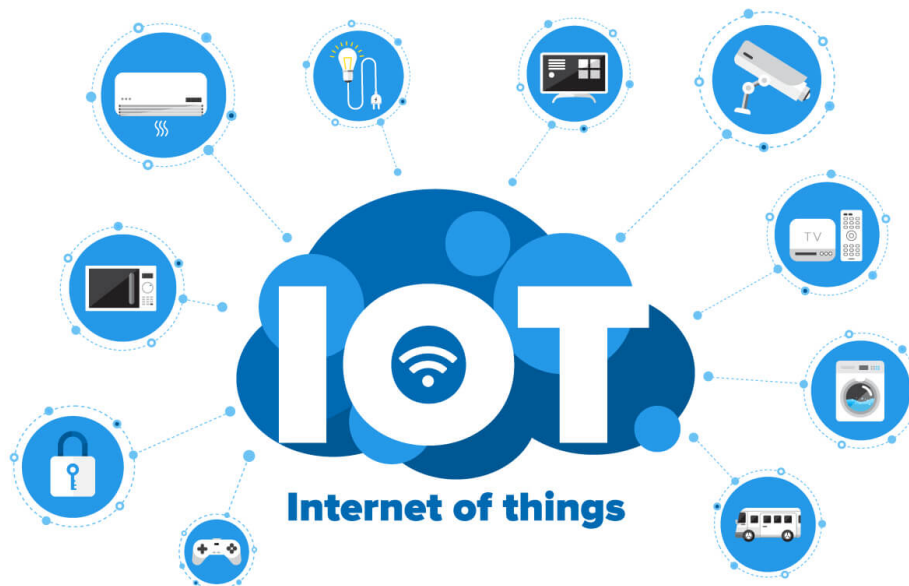


Motor Driver & Modular Code (OOP)

Third Project



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents the upgrade of the Project 2 Motor Protection System into a modular, object-oriented firmware built around an L298N dual H-bridge motor driver, replacing the simple on/off relay used previously. The relay could only switch the motor on or off; the L298N adds genuine speed control (PWM) and direction control (forward / reverse / Stop / brake), while keeping every protective feature from Project 2: continuous current monitoring, an Emergency Stop, and status LEDs.

Beyond the hardware upgrade, the single-file `main.cpp` from Project 2 was refactored into six files split strictly along the boundaries required by the assignment: a central `config.h`, a **C-style** current sensor driver (`CurrentSensor.h/.cpp`), a C++ class motor driver (`L298N.h/.cpp`), and a slim `main.cpp` that only wires the pieces together. Several professional-grade improvements were added on top of the assignment's minimum requirements – an interrupt-driven E-Stop, a two-stage current filter, and active braking – all detailed in Section .

The project also surfaced a set of hardware/software debugging challenges that took real effort to resolve: a microcontroller pin-mapping mismatch between the assumed board and the actual ESP32-S3 board in use, and a PWM duty-cycle issue that silently prevented the motor from starting. Both are documented in detail in Section , since the debugging methodology used there is arguably as valuable a takeaway as the working code itself.

2. Hardware Architecture & Pin Mapping

The project employs an ESP32 DevKit v1 development board, identical to the one used in Project 2. All digital and analog peripherals are powered from the board's 5 V logic rail, with the exception of the current sensor, which is supplied by the ESP32's 3.3 V pin. The motor is driven via the L298N using an independent 5 V bench power supply; the driver's motor supply rail is explicitly separate from the ESP32's logic power rail.

Design change: relay removed

Project 2 used a relay purely as an on/off switch for the motor. The L298N's `stop()` (coast) and `brake()` (active short) methods make the relay's job redundant – the driver can already fully de-energize or hard-stop the motor without any additional hardware. The relay and its GPIO were removed from the design entirely rather than kept alongside the L298N, avoiding an unnecessary extra failure point and pin.

PWM configuration

The motor is driven at a fixed carrier frequency chosen to stay above the audible range while remaining compatible with the L298N's switching characteristics:

- **PWM frequency:** 20 kHz (`PWM_FREQ_HZ`)
- **PWM resolution:** 8-bit, i.e. duty values 0–255 (`PWM_RESOLUTION_BITS`)

Peripheral	Peripheral Pin	ESP32-S3 GPIO	Electrical Logic / Notes
L298N Motor Driver	IN1	GPIO 25	Digital Output, direction control
L298N Motor Driver	IN2	GPIO 26	Digital Output, direction control
L298N Motor Driver	ENA	GPIO 27	PWM Output, 20 kHz / 8-bit (0–255) speed control
L298N Motor Driver	12V / GND	5V source	Motor supply from external 5V bench source, common ground
Current Sensor (ACS712)	OUT	GPIO 4	Analog Input, ADC1-capable on ESP32-S3
Current Sensor (ACS712)	VCC / GND	3.3V / GND	Separate ground reference than other components
E-Stop Push Button	Signal	GPIO 37	Digital Input, INPUT_PULLUP, hardware interrupt on FALLING edge
LED – Running (Green)	Anode (+)	GPIO 39	Digital Output through current-limiting resistor
LED – Stopped (Yellow)	Anode (+)	GPIO 40	Digital Output through current-limiting resistor
LED – High Current (Red)	Anode (+)	GPIO 41	Digital Output through current-limiting resistor
LED – Low Current (Blue)	Anode (+)	GPIO 38	Digital Output through current-limiting resistor
All LED Cathodes	(–)	GND	Common ground reference

Table 1: ESP32-S3 pin mapping for the motor driver & modular code system.

- **Default run duty cycle:** 255/255 (100%) – see Section for why a lower duty cycle (180/255) was tried first and failed.

3. Schematic and Circuit

Schematic

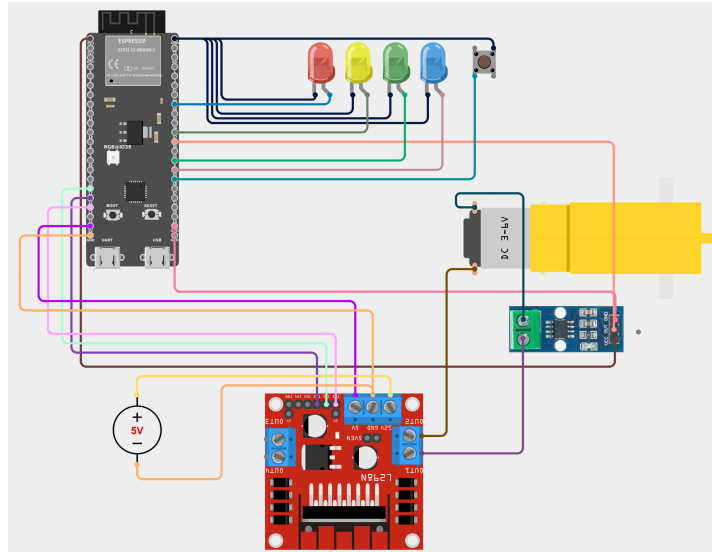


Figure 1: wiring schematic

Circuit

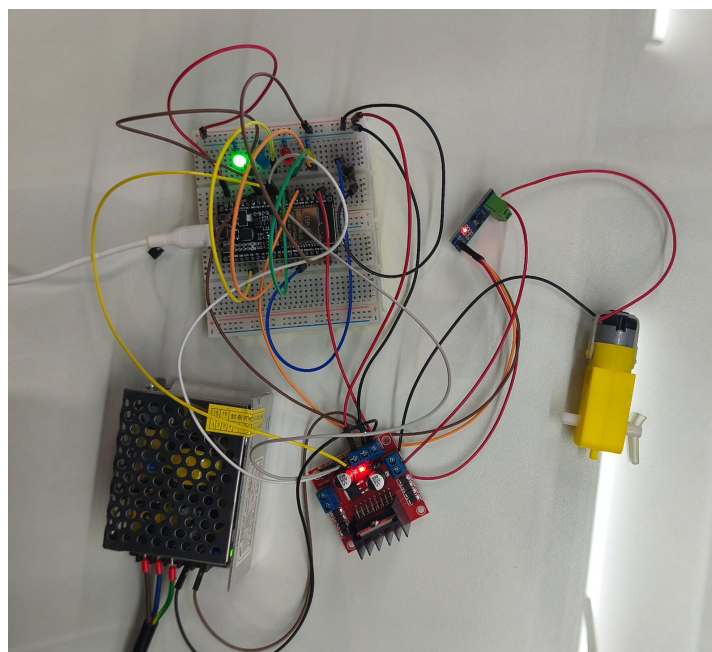


Figure 2: Assembled circuit under test

4. Modular Software Architecture (OOP Design)

The assignment's core requirement was splitting a single-file sketch into independent, purpose-built modules. The resulting file structure is:

File	Style	Responsibility
<code>config.h</code>	Namespace of <code>constexpr</code>	Every pin, PWM setting, threshold, and timing constant in the project. No other file contains a magic number.
<code>CurrentSensor.h/.cpp</code>	C-style free functions	Reads and filters the analog current sensor; internal state hidden in an anonymous namespace instead of a class, per the assignment's explicit requirement.
<code>L298N.h/.cpp</code>	C++ class	Encapsulates the motor driver: private pins and state, public <code>forward()/reverse()/stop()/brake()</code> interface.
<code>main.cpp</code>	Application logic	Wires the modules together, owns the system state machine, the E-Stop interrupt, and the LED outputs.

Why `config.h` uses a namespace instead of `#define`

Rather than the traditional `#define PIN_X 25` style, every constant lives inside `namespace Config { ... }` as a `constexpr`. This gives the same "one central place for every setting" benefit the assignment asks for, but with compiler type-checking and proper scoping – a constant like `Config::CURRENT_LIMIT_A` cannot silently collide with an unrelated macro elsewhere in the codebase the way a bare `#define` could.

Why `CurrentSensor` stays C-style but still avoids raw globals

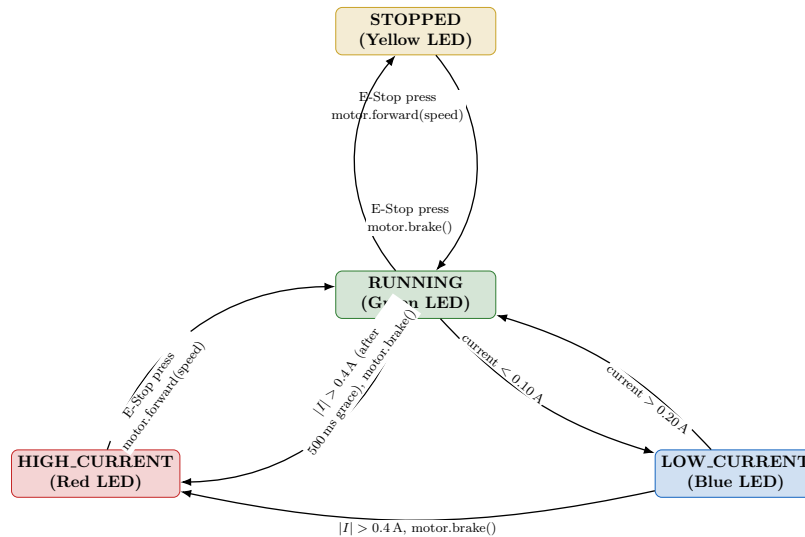
The specification is explicit that the current sensor driver must *not* be a class. To respect that while still writing defensible code, all sensor state (`s_pin`, `s_zeroVoltage`, `s_filteredAmps`, ...) is declared inside an anonymous namespace at the top of `CurrentSensor.cpp`. This is the C++ equivalent of C's `static` file-scope globals: the three required functions (`CurrentSensor_Init`, `CurrentSensor_ReadAmps`, `CurrentSensor_Is`) are the only way anything outside this file can touch that state.

Why `L298N` is a genuine class, not just a wrapper

The `L298N` class stores its three pins and current `MotorDirection/speed` as **private** members, exposing only the five methods the assignment specifies: `L298N(in1, in2, ena)`, `begin()`, `forward(speed)`, `reverse(speed)`, `stop()`, plus `brake()`. Because the state is private, `main.cpp` can query `motor.getDirection()` or `motor.getSpeed()` without ever risking an inconsistent internal state – every transition happens through one of the five public methods, each of which updates direction and PWM duty atomically.

5. System State Machine

The firmware keeps the same four-state design philosophy as Project 2, now driving the L298N instead of a relay.



Key differences from Project 2's state machine

- **Interrupt-driven, not polled.** The E-Stop transition is triggered by a hardware interrupt (`attachInterrupt`, FALLING edge) instead of being polled every `loop()` iteration. See Section .
- **Active brake, not coast.** Both the E-Stop-from-RUNNING transition and the overcurrent trip call `motor.brake()` (both H-bridge inputs driven HIGH, shorting the motor windings) rather than `stop()` (coast), stopping the motor considerably faster.
- **LOW_CURRENT is informational, not a fault.** The motor keeps running in LOW_CURRENT – only HIGH_CURRENT latches and requires a reset.

6. Troubleshooting & Challenges

This section documents the real debugging path followed during the build, since each issue below cost meaningful time and taught something specific.

6.1 Isolating the motor from the rest of the system to debug it

When the motor first failed to turn under the full firmware, the fault could have been in the wiring, the driver, the PWM configuration, or the state machine logic – four independent suspects. Rather than debugging all of them at once inside the full sketch, a minimal, standalone test sketch was written that only initializes the L298N and commands `forward()` / `reverse()` in a timed loop, with no button, no current sensor, and no state machine involved at all. This is a deliberate engineering technique: isolate

the smallest subsystem that can be independently verified, confirm it works in isolation, then reintroduce the rest of the system one piece at a time. Running this isolated test immediately confirmed the L298N, its wiring, and the chosen GPIOs (25/26/27) were all correct – which narrowed the remaining search space considerably before the real cause was found (Section 6.3).

6.2 PWM duty cycle too low to overcome motor stiction

Even with the driver confirmed working, the motor would not start when commanded at a duty cycle of 180/255 ($\approx 70\%$) – yet the identical code commanding 255/255 (100%) started it reliably every time.

Root cause: a motor requires enough voltage/torque to overcome *static friction* (stiction) in its brushes and gearbox before it can begin moving at all; once already spinning, *kinetic* friction is lower, so less drive is needed to keep it going than to start it. Two factors stacked against the 180/255 duty cycle in this build:

1. The L298N itself drops roughly 1.5–2 V across its internal transistors, so a nominal 5 V motor supply was already delivering meaningfully less than 5 V to the motor terminals before any PWM scaling was applied.
2. At 70% duty on top of that already-reduced voltage, the effective starting torque fell below the threshold needed to break static friction, so the motor drew current (visible on the sensor) but never physically moved.

Fix: `Config::DEFAULT_RUN_SPEED` was raised to 255. A more general fix for future work – letting the motor run at a lower, quieter cruise speed without this problem – is a short *kick-start ramp*: briefly command 100% duty for ~ 100 – 200 ms to break stiction, then drop to the desired lower cruise duty once the motor is already turning (see Section).

6.3 Current sensor is still ratiometric to its supply (carried over from Project 2)

As in Project 2, the ACS712's zero point and sensitivity both scale with its supply voltage rather than being fixed constants. The boot-time `CurrentSensor_Calibrate()` routine (300 averaged samples, motor guaranteed off) continues to correct for the zero-point drift automatically. The sensitivity constant (`CURRENT_SENSITIVITY_V_PER_A` = 0.066, the 5 V datasheet figure) should still be re-derived if the sensor's actual supply differs from 5 V, exactly as noted in the Project 2 report – this remains an open item for absolute (not just relative/threshold) current accuracy.

7. Professional-Grade Additions

Beyond the assignment's minimum requirements, the following were deliberately added to make the firmware closer to production-quality embedded code:

1. **Interrupt-driven E-Stop instead of polling.** The button is attached via `attachInterrupt(..., FALLING)`. The ISR (`IRAM_ATTR onEstopChange()`) does the absolute minimum: a timestamp-based debounce check and setting a `volatile bool` flag. All real work – motor control, Serial output, state transitions – happens in `loop()`, never inside the ISR, which is the correct pattern for keeping interrupt handlers short and avoiding reentrancy bugs.

2. **Two-stage current filtering.** Each reading first *oversamples* 32 raw ADC conversions (`CURRENT_OVERSAMPLE_COUNT`) to reduce quantization noise, then passes the result through an **Exponential Moving Average** low-pass filter (`CURRENT_EMA_ALPHA = 0.2`) to smooth transient spikes/PWM switching noise, so the overcurrent trip reacts to sustained draw rather than momentary blips.
3. **Type-safe, centralized configuration.** All constants live in a namespace `Config` of `constexpr` values rather than `#define` macros, giving compiler-checked types and proper scoping.
4. **Encapsulated state even in the C-style driver.** The current sensor's internal variables are hidden inside an anonymous namespace in `CurrentSensor.cpp` rather than exposed as true globals, honoring the "no class" requirement while still avoiding namespace pollution.
5. **Active braking on both E-Stop and fault trip.** `brake()` (both H-bridge inputs HIGH, shorting the motor) is used instead of `stop()` (coast) whenever the system needs to halt the motor immediately, which brings a spinning motor to rest meaningfully faster – important for both the E-Stop's responsiveness and the overcurrent trip's protective intent.
6. **Hysteresis on the LOW_CURRENT/RUNNING boundary.** A deadband around `LOW_CURRENT_THRESHOLD_A` prevents the Blue/Green LEDs from flickering back and forth when the current reading sits close to the threshold.
7. **Fail-safe boot sequence preserved from Project 2.** The motor is forced to `stop()` *before* the current sensor calibrates, guaranteeing the zero-current baseline is measured with the motor definitely off.

8. Bill of Materials

Item	Qty	Notes
ESP32 DevKit v1 development board	1	Main controller
L298N dual H-bridge motor driver module	1	Replaces Project 2's relay; provides speed + direction control
ACS712 current sensor module (30A variant)	1	Sensitivity 66 mV/A datasheet value; ratio-metric to supply, see Sec. 6.4
Brushed DC gear motor (3–6 V)	1	Load under test
Momentary push button	1	Wired to <code>INPUT_PULLUP</code> , interrupt-driven
LEDs: green, yellow, red, blue	1 each	5mm standard LEDs
220 Ω resistor	4	One per LED
5V bench power supply	1	Feeds L298N's 12V/GND terminal block
Breadboard + jumper wires	–	–

9. What I Learned

Modular code makes debugging dramatically faster

Splitting the firmware into `config.h`, `CurrentSensor`, and `L298N` was originally framed as a code-organization exercise, but it paid off directly during debugging: because `L298N` is a self-contained class with no dependency on the current sensor or the button, it was possible to write a five-minute standalone test sketch that exercised *only* the motor driver and prove it worked, without the sensor or state machine as confounding variables. A monolithic single-file sketch would have made that kind of isolated testing far more awkward.

PWM duty cycle has a real physical floor, not just a linear scale

It is tempting to treat PWM duty cycle as a purely linear speed knob, but a motor's required *starting* duty cycle can be meaningfully higher than its required *running* duty cycle, because static and kinetic friction are not the same. A system that needs to reliably start a motor at a low cruise speed should not just set a low constant duty cycle – it needs a brief full-power kick-start before settling to the lower target (Section).

Other takeaways

- Interrupts are the correct tool for a hard real-time requirement like E-Stop responsiveness – polling in `loop()` adds latency proportional to whatever else is happening in that iteration.
- Keeping an ISR to "set a flag and return" avoided any risk of `Serial` calls or motor commands running inside interrupt context.
- A two-stage filter (oversample + EMA) was noticeably more stable against PWM switching noise than either technique alone would have been.

11. Future Improvements

- **Kick-start ramp in `L298N::forward()/reverse()`:** briefly drive at 255/255 for ~150ms, then step down to the requested cruise speed, so lower speeds can be commanded without risking a stiction-stall like the one in Section .
- **Second motor channel:** the `L298N` has a second H-bridge (OUT3/OUT4) entirely unused in this build; the existing `L298N` class could be instantiated a second time with zero code duplication.
- **Re-derive current sensitivity for the actual supply voltage** used in the final build, so the printed amps value is absolute/calibrated rather than only reliable as a relative trip threshold.
- **Non-volatile fault logging:** persist the last trip's current value and timestamp (e.g. to flash/NVS) so a fault can be diagnosed after power loss, not just read live off the Serial Monitor.